SiteSense

AI-powered Retail Expansion Intelligence Platform · Pune

Triwizathon · Problem Statement 1
Suyash Patil · Harshad Khatale · Tanuja Saravale · Madanaj Sagar
Walchand College of Engineering

1. The problem statement

Problem Statement 1 - Retail Expansion Intelligence Platform. Choosing the right place for a new store is hard. A location may have many people but also strong competition, high rent or poor access. Doing this by manual research or gut feeling leads to poor investments and missed chances.

In simple words

A retail chain wants to open its next shop. There are thousands of possible places in a city. **Which place will earn the most, which are risky, and why?** We must build software that answers this with data and AI, and explains its answer.



Figure 1. A location is never good or bad on one factor. The platform must weigh all of these together.

What the challenge asks, and where we answer it

The challenge asks for	Our answer in SiteSense
Use geographic and demographic data, competitors, existing stores, access and operating costs	Real OpenStreetMap + WorldPop data, 35 locality prices, metro/bus/road access, your own stores, estimated rent
Estimate customer demand and predict commercial viability	Gravity-model simulation (demand → sales → profit) and a viability model learned from where chains already trade
Rank candidate locations	Opportunity index + learned ranker, ordered by the strategy you choose
Evidence-based explanations: strengths, risks, comparison with alternatives	Top reasons, risk flags, confidence level and a “why #1 beats #2” sentence for every site
AI must do real analysis (not just a chatbot)	Five trained or optimised models do the work; language is only used to read your request, never to invent answers
State assumptions, data sources and limitations	Every number is labelled real, modelled or estimated; a Data & trust page lists sources, assumptions and model scores

5	6,541	15,350
business types	hexagon zones (≈10 ha each)	real map features used

2. Our solution

SiteSense tells a retail owner where to open the next store. It splits Pune into small hexagons, measures what surrounds each one, learns what successful locations look like, simulates how customers would choose between stores, and explains every recommendation in plain language. The owner picks a business type and limits; the app does the rest.



Figure 2. The six steps from raw data to an explained recommendation.

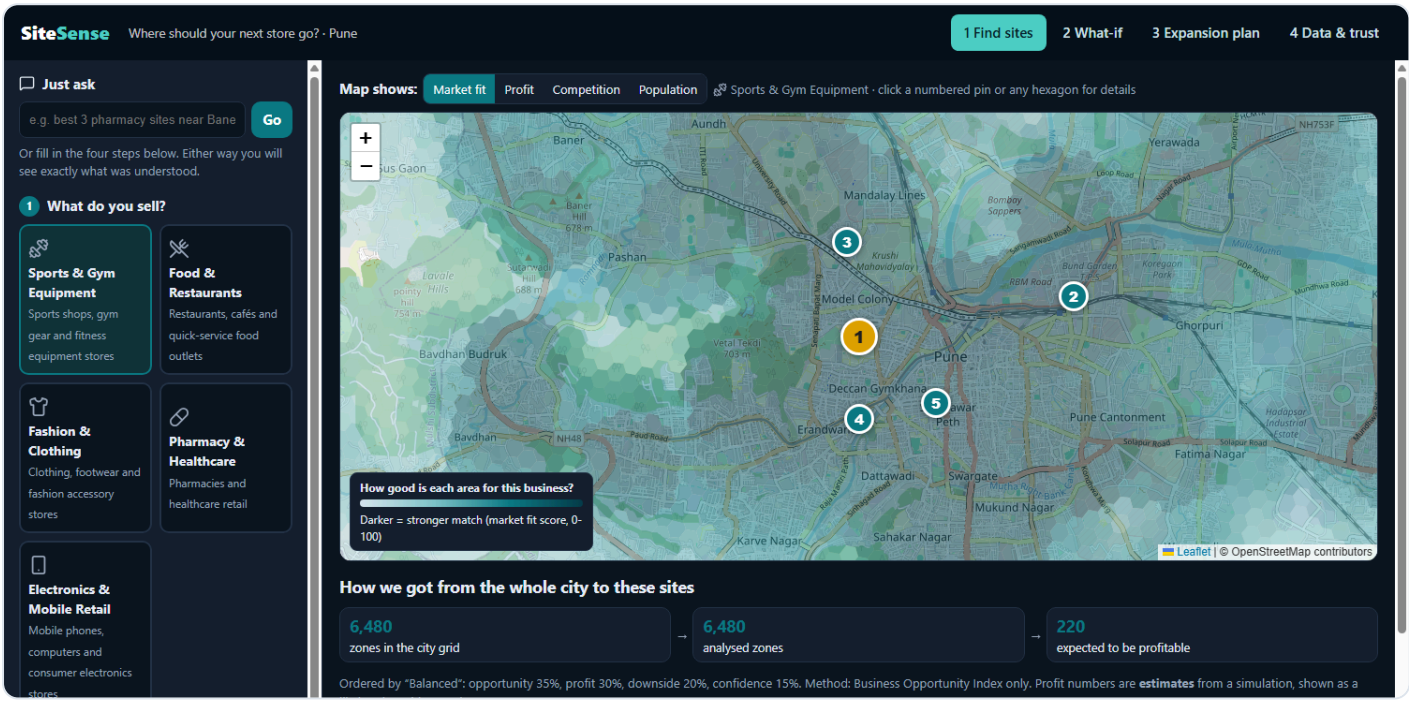


Figure 3. The working app: guided steps (left), hexagon map with ranked pins, and plain-language results.

What the owner does

1. **Choose** the business (or just type: “best 3 pharmacy sites near Baner under 60 lakh”).
2. **Set limits**: start-up money, highest rent, area, store size.
3. **Pick what matters**: balanced, max profit, play it safe, or best market fit.
4. **Add existing stores** by clicking the map, so new sites do not steal their sales.
5. **Read the answer**: top sites with a weak / likely / strong profit range, reasons, risks and confidence.
6. **Test it**: what if rent rises 30%, or a rival opens next door? Or plan 2-6 stores together.

Why it is different

- **Honest ranges**, never a single made-up number.
- **Explainable**: each site lists its top reasons and risks.
- **Category-aware**: a pharmacy and a fashion store need different things (the “Retail DNA”).
- **Transparent data**: every value says if it is real or estimated.

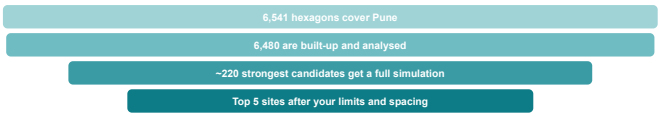


Figure 4. How the engine narrows the whole city to a short list (shown live in the app).

2.1 How the AI works

There is **no sales data** for any shop in the area, so we avoid pretending. We learn from *where established chains already trade* (a weak but real signal), test the models on whole neighbourhoods they never saw, and simulate sales with stated assumptions.

Part	What it does	Technique
Opportunity index (BOI)	Scores every zone 0-100 on 10 factors; the weights differ per business (the "Retail DNA").	Simple weighted formula - fully explainable
Viability model	Learns what proven locations look like. "Proven" = an established chain outlet already trades in the zone.	XGBoost, positive-unlabeled learning
Location ranker	Orders zones from best to worst for each business; higher rent never raises a score.	LightGBM LambdaRank
Rent model	Spreads 35 known locality rents to every zone, with a likely range.	CatBoost + validation on unseen localities
Profit simulation	Shoppers split their spending between stores by distance and attractiveness; repeated 300 times with uncertain inputs.	Huff gravity model + Monte Carlo
Cannibalisation	Estimates how much of a new store's sales would come from your own nearby stores.	Same gravity model
Explanations	Turns model contributions into plain reasons, risk flags and a confidence level.	SHAP + rules
Zone types	Groups areas into 6 types (office hub, student belt, high street…).	K-means on PCA
Expansion planner	Chooses the best set of stores together, within budget and spacing.	OR-Tools CP-SAT
Ask in English	Turns a sentence into a safe, whitelisted search.	Rule-based parser (no chatbot)

The "Retail DNA": what matters for each business



Figure 5. Weights (0-100, each column adds up to 100). Darker = matters more. Stored in a config file, shown in the app.

How a score and a profit are made

Opportunity score = Σ weight × factor score ÷ 100, for the 10 factors.
Bands: Prime 80+ · Strong 65-79 · Viable 50-64 · Marginal 35-49 · Avoid <35

Profit = sales × margin − rent − fixed costs.
Sales = what each nearby neighbourhood spends × the share this store wins against rivals, summed over the catchment. Repeated 300 times with varied spend, margin, rent, distance sensitivity and number of unmapped stores. We report the 10th, 50th and 90th percentile and the chance of profit.

Safeguards: the defining chain outlet is hidden from the model's inputs (no cheating); every zone is scored by a model that never saw its neighbourhood; rent can only lower a score; a plain-index fallback is used when labels are too few.

2.2 Feature counts

17	product features a user can use (list on the right)
31	measured or estimated values stored for every zone (people, workers, students, prices, rent, access, growth…)
41	values per zone and business (competition, complementors, 10 factor scores, index, profit range, model scores)
10	factor groups (FG1-FG10) that feed the opportunity index
19	inputs the viability and ranking models learn from (18 zone features + the viability score)
9	AI components built (of 11 in the plan; the fast surrogate model and a learned growth forecaster are left for later)

The 17 product features

1 Ask in plain English · **2** 5 business types · **3** Your limits · **4** 4 strategies · **5** Your existing stores · **6** Brand strength · **7** Hexagon map, 4 layers · **8** Ranked top sites · **9** Reasons for each site · **10** Risks and confidence · **11** Why #1 beats #2 · **12** Zone profile · **13** Unit economics · **14** What-if · **15** Expansion plan · **16** Briefing · **17** Data and trust page

3. Datasets and features

Data sources

Source	Size	Used for	Type	Licence
OpenStreetMap (Overpass API)	15,350 map features in 10 queries	Shops, amenities, offices, healthcare, leisure, tourism, bus stops, rail, roads	Real	ODbL
WorldPop 2020, India 1 km raster	1 raster (19.1 MB) → 6,917,973 people over the area	Residents per zone (then spread by OSM activity)	Modelled	CC BY 4.0
ohsome API (HeiGIT)	history of mapped shops, 2021 → 2026	Area growth trend	Real (service was down at build time; flagged)	ODbL
Locality price table	36 Pune localities listed (35 fall inside the grid)	Property price per sq ft → affluence and rent	Estimate (our table)	Own
Metro stations list	28 stations (+145 OSM rail)	Distance to metro / rail	Curated + real	Own / ODbL
Category settings (YAML)	5 businesses	Margins, fixed costs, store size, spend per person, chain names	Assumptions	Own
Basemap tiles	OpenStreetMap standard	Map background	Real	ODbL

What was collected from OpenStreetMap

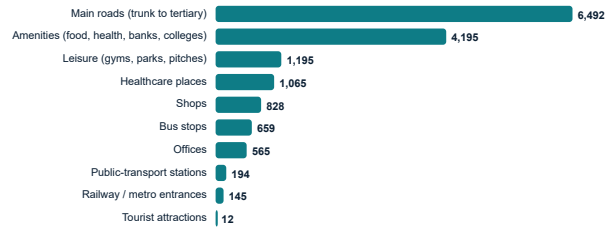


Figure 6. Map features fetched for the study area (15,350 in total).

The study area

Area covered	Pune, Pimpri-Chinchwad, Hinjewadi, Kharadi (rectangle 18.44-18.66°N, 73.72-73.97°E)
Zone shape and size	H3 hexagons, resolution 9, about 10.5 hectares each
Zones	6,541 total, 6,480 built-up and analysed
Population in area	6.92 million (WorldPop 2020)
Median residents within 1 km	20,117
Local property prices	Rs 5,100 to 12,803 per sq ft (smoothed estimate)
Estimated shop rent	Rs 50 to 139 per sq ft per month
Zone types found	6 (office hub, student belt, dense residential, transit node, high street, emerging periphery)
Build time	about 5 minutes from cached downloads; stored results about 11 MB

Data by business type

Business	Competitors mapped	National / regional chains	Chain names listed	Proven (positive) zones	Ranking method
Sports & Gym Equipment	25	6 / 1	8	7	Index only (too few labels)
Food & Restaurants	1,904	148 / 44	14	103	Learned models + index
Fashion & Clothing	352	71 / 39	19	52	Learned models + index
Pharmacy & Healthcare	219	29 / 2	9	25	Learned models + index
Electronics & Mobile Retail	139	27 / 15	12	35	Learned models + index

4. Tech stack

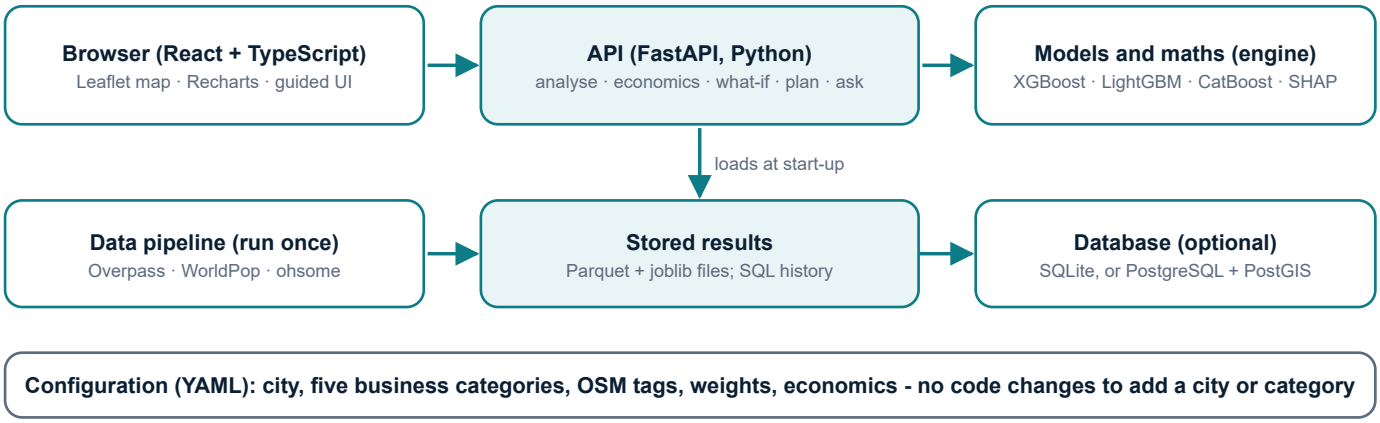


Figure 7. Architecture. The slow work (downloading, training, simulating all zones) runs once; the app then answers in seconds.

Layer	Technology	Why we chose it
Frontend	React, TypeScript, Vite, Leaflet, h3-js, Recharts, Lucide icons	Fast, typed UI; Leaflet draws the hexagon map; h3-js converts zone ids to shapes
Backend	Python, FastAPI, Pydantic, Uvicorn	Validated inputs, automatic API docs, easy to run with the ML code
Data and geo	Pandas, NumPy, SciPy, Uber H3, Rasterio, PyArrow (Parquet)	Hexagon grid, raster reading, compact fast storage
Machine learning	XGBoost, LightGBM, CatBoost, scikit-learn, SHAP	Strong on small tabular data; SHAP gives explanations
Optimisation	Google OR-Tools (CP-SAT)	Picks the best group of stores under budget and spacing
Storage	SQLite by default; PostgreSQL + PostGIS optional (SQLAlchemy, GeoAlchemy2)	Run history and spatial queries; zero set-up locally
Delivery	Docker Compose, YAML config, optional LLM key (rephrases only)	One-command run; add a city or business by editing YAML

5. Feasibility

Technical feasibility: it already runs

- Full pipeline works end to end on real Pune data and builds in about 5 minutes.
- A search finishes in seconds on a laptop; unit tests cover the gravity maths (more rivals never raise sales, own stores cause overlap).
- Only free tools and open data are used - no paid API is needed.

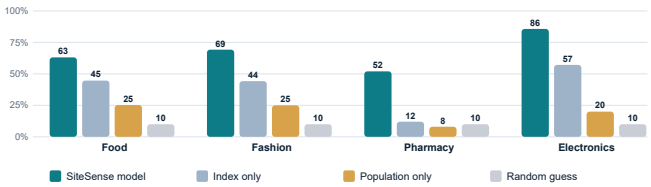


Figure 8. Share of proven zones found in the top 10% (higher is better). Random guessing scores 10%. Sports has too few labels to test.

Does the learning work? (tested on unseen neighbourhoods)

“Of the zones that really host an established chain, how many land in the model’s top 10%?”

The learned model beats a population-only rule and a plain index in every business it was tested on (e.g. pharmacy 52% vs 8% for population only). These checks show the model finds *chain-like* locations; they are not proof of profit.

5.1 Viability: would it be useful and sustainable?

Who needs it and why

- **Retail chains and franchise owners** who open stores every year and today rely on brokers and gut feeling.
- **Small entrepreneurs** choosing a first shop, who cannot afford consultants.
- **Real-estate and mall teams** who want to judge which tenant fits a location.

It saves research time, makes trade-offs visible (rent vs footfall vs rivals) and shows *why*, so decisions can be defended to investors.

How it can sustain itself

- Software subscription per chain or per city.
- Paid reports for a single location decision.
- Premium data upload: a client's real sales make the model far better.

Why it can scale

- A new city = one YAML file and a rebuild; a new business = one block of weights, tags and economics.
- Open data, no per-user data cost; all heavy work is pre-computed.

Honest limitations

- **No real sales data.** Profit comes from a simulation with our assumptions (spend per person, margins, store density). It is decision support, not a guarantee.
- **"Proven" means a chain is there**, not that it is profitable.
- **OpenStreetMap is incomplete.** Only 25 sports shops are mapped in Pune, so sports uses the plain index and shows lower confidence.
- **Rent model is basic.** On unseen localities it is off by about 15%, similar to a simple price rule (14%).
- **Growth data** was unavailable at build time, so the growth factor is neutral and flagged.
- Population is a modelled 1 km raster; prices come from a 35-locality table; the study area is a rectangle (not the exact city border); no zoning, lease or online-demand data.

Next steps

- Clip the grid to the real city boundary; retry growth data.
- Calibrate with real sales from one pilot chain.
- Add live rent listings and footfall data; more cities.
- Fast surrogate models, a learned growth forecaster, user accounts and PDF reports.

6. References

1. Triwizathon, *Problem Statement 1: Retail Expansion Intelligence Platform*.
2. Huff, D. L. (1963). A probabilistic analysis of shopping center trade areas. *Land Economics*, 39(1). Reilly, W. J. (1931). *The Law of Retail Gravitation*.
3. Lundberg, S. & Lee, S.-I. (2017). A unified approach to interpreting model predictions (SHAP). *NeurIPS*.
4. Chen, T. & Guestrin, C. (2016). XGBoost. *KDD*. Ke, G. et al. (2017). LightGBM. *NeurIPS*. Prokhorenkova, L. et al. (2018). CatBoost. *NeurIPS*.
5. Burges, C. (2010). From RankNet to LambdaRank to LambdaMART. *Microsoft Research TR-2010-82*.
6. Elkan, C. & Noto, K. (2008). Learning classifiers from only positive and unlabeled data. *KDD*.
7. Roberts, D. R. et al. (2017). Cross-validation strategies for data with spatial structure. *Ecography*, 40(8).
8. OpenStreetMap contributors, ODbL 1.0, openstreetmap.org/copyright; Overpass API; ohsome API (HeiGIT).
9. WorldPop (2020). Gridded population, India 1 km, CC BY 4.0, worldpop.org.
10. Uber H3 hexagonal index, h3geo.org. Google OR-Tools CP-SAT. FastAPI, React, Leaflet, scikit-learn, PostGIS.
11. Government of India, Digital Personal Data Protection Act 2023 (no personal data about shoppers is used).

Map data © OpenStreetMap contributors. All figures in this document are produced by the project's own build (2026-10-24 15:59:08+00:00 UTC) and are estimates unless marked real.